

Characterizing Human-Agent Dynamics in Agent-authored Pull Requests

Anonymous Author(s)

Abstract—Pull requests (PRs) are becoming the primary interface through which coding agents contribute to software repositories, yet how these contributions are actually reviewed remains poorly understood. Existing evaluations largely reduce agent-authored PRs to a single endpoint—merged or not—collapsing fundamentally different review histories into the same label: a PR integrated without human attention looks identical to one that underwent several rounds of substantive revision. We argue that agent-authored PRs must instead be studied as workflows, and reconstruct four stages—Entry, Feedback, Feedback-Response Cycles, and Integration—from public GitHub timelines across a same-window corpus of 325,523 agent-authored PRs, with human and bot PRs as context, linked to comments, reviews, commits, actor metadata, and outcomes. Our analysis shows that most agent-authored PRs never enter substantive review at all, and when they do, the reviewer is nearly as likely to be another agent as a human, meaning first-line triage of agent-generated code is itself increasingly automated. Decomposing PRs into workflow pathways further reveals that the vast majority follow one of two shallow trajectories (no feedback or feedback with no code response), while the minority that undergo iterative revision behave in systematically different, more consequential ways. Interestingly, the PRs with the cleanest-looking review outcomes are often those with the least actual iteration, suggesting that visible approval signals can mask an absence of scrutiny rather than confirm its presence. These findings show that merge rate and review presence are poor proxies for how agent-authored contributions are actually vetted and argue for treating PR-thread workflow signals as first-class evaluation criteria for coding agents.

Index Terms—AI Agents, Pull Requests, Code Review, HCI

I. INTRODUCTION

Coding agents are moving from local completion and chat assistance toward repository-facing tools that can plan tasks, edit multiple files, run checks, and open pull requests (PRs) on GitHub [1]–[3]. This shift relocates where agent work becomes accountable because a patch produced by an agent is not project state until it passes through the repository’s PR workflow. The PR thread is therefore the gate at which human developers and an agent’s output actually meet.

Yet we still evaluate that output almost entirely by its endpoint. Coding-agent benchmarks and recent studies of agent-authored PRs report task completion, benchmark accuracy, code change, testing, security, reviewer engagement, review loops, acceptance, and merge rates [4]–[14]. These measures matter, but a merge label collapses everything happening between PR creation and integration. The same “merged” outcome can describe a PR that received no human attention and was integrated silently, one that made a short exchange followed by a fix, or several rounds of revision. These cases are identical as outcomes, but not as process.

Opening the thread, however, is not straightforward. PRs are socio-technical workflows shaped by technical risk, contributor history, project norms, and reviewer workload [15]–[17], and agent-authored PRs add new measurement hazards. Feedback may originate from humans, agents, or automation bots, and raw PR-thread messages may mix review-relevant feedback with deployment, status, helper, or check boilerplate. Characterizing how agent PRs are handled therefore requires reconstructing the PR timeline as layered evidence rather than merely counting whether a PR was reviewed or merged.

We take this view and study agent-authored PRs as feedback-response-revision-integration workflows. From public PR timelines, we distinguish raw visible feedback from retained review-relevant feedback, identify who first engages, summarize feedback body form and intent, reconstruct feedback-response cycles, and relate the resulting workflow pathways to integration endpoints. The contribution is not a new endpoint predictor or a new isolated PR feature family. It is a PR-level reconstruction of how entry, feedback, actor, post-feedback code activity, and endpoint compose in the same public workflow.

We assemble a large-scale, same-window corpus of 325,523 agent-authored PRs from 2025-04-01 to 2026-01-31, together with 29,808 human- and 1,206 bot-authored PRs as same-window context, and link each PR to its comments, reviews, commits, actor metadata, and outcome. Four research questions (RQ) drive our analysis.

- **RQ1 (Entry).** How often do agent-authored PRs enter PR-thread interaction, and who enters first?
- **RQ2 (Feedback).** Where is feedback delivered, who provides it, and what body forms and intents appear?
- **RQ3 (Feedback-response cycles).** What workflow pathways, cycle latencies, and responses follow feedback?
- **RQ4 (Integration).** How do entry, feedback, and response cycles compose with integration outcomes?

Our results show that agent-authored PRs follow no single review path, and that the endpoint hides most of it. Only 26.6% of marked agent PRs receive any feedback, while retained review-relevant feedback remains a minority across strict, main, and relaxed filtering boundaries (10.3–13.3%, with 10.5% under the main definition), and 10.3% contain at least one feedback-response cycle. Among retained-feedback PRs, the first responder is nearly evenly split between humans (50.0%) and agents (47.4%), with automation bots accounting for 2.5%. In other words, close to half of retained-feedback first-line triage of agent work is itself performed by other

agents. The derived pathways make the heterogeneity concrete because most agent PRs have no feedback (73.4%) or feedback with no code response (13.6%), while other pathways are smaller but behaviorally distinct. Post-feedback revision is sparse overall yet concentrated in a heavy tail of cycle-bearing PRs. Tellingly, late-feedback PRs, i.e., those whose final thread event is approval-like or merge-adjacent, show the highest merge rate, meaning the cleanest "successful review" outcomes often involve the least iteration.

These findings argue that coding-agent evaluation should report PR-thread workflow signals, including feedback entry, retained feedback, first-entry actor, feedback body form, feedback intent, response timing, post-feedback revision, and integration endpoint, instead of reducing them to pass, review, or merge labels. Our contributions are as follows.

- 1) **In-the-wild PR workflow corpus.** We construct a large-scale, same-window dataset linking agent-authored PRs, human PRs, PR-thread comments, feedback bodies, actor metadata, and integration outcomes. The accompanying repository provides the analysis scripts, generated tables and figures, source maps for reported numbers, and a 50% random sample of the dataset for verification.¹
- 2) **Human-agent PR workflow framework.** We characterize agent-authored PRs as an Entry, Feedback/Response Cycle, and Integration process, with measures for triager, feedback actors, body form, cycle timing, response, and ending, plus intent labels for aggregate feedback summaries.
- 3) **Empirical characterization of agent-authored PR workflows.** We characterize how agent-authored PRs enter feedback, who first triages them, how feedback channels, actors, body forms, and intents differ, how much code response follows, and how workflow pathways relate to integration outcomes.

II. METHODOLOGY

A. Data Collection & Structure

To collect the data, we follow a scraping procedure similar to prior work [8]. Our study window is 2025-04-01 to 2026-01-31. We use GitHub GraphQL to collect PR-level actions and linked metadata for agent-, human-, and bot-authored PRs. In this paper, *agent-authored* refers to PRs attributed by visible public signals in Table I [6]–[8]. When multiple signals match, account- and branch-based signals take precedence over text matches. The unit of analysis is, hence, the PR timeline reconstructed from PR actions.

Feedback-actor classes are signal-based account labels. *Human* means no agent or automation bot signal remains after refinement. *Agent* means the normalized login matches coding-agent or review-agent patterns. *Automation bot* means the login matches explicit bot markers or service-automation patterns such as dependency, CI, coverage, quality, deployment, CLA, and merge-queue services. This label does not require GitHub’s account type to be `Bot` and does not indicate

whether a service uses AI internally. Figures shorten these classes to Human, Agent, and Bot, with Bot abbreviating automation bot.

The pre-scope collection by tracking group and its repository-visible attribution signals are summarized in Table I. In this table, PRs are distinct canonical repository–PR keys, repositories are distinct base repositories, and commits, comments, and reviews are distinct GitHub event identifiers. Some PRs can match multiple repository-visible agent signals, so agent rows are not additive. The total row reports the unique union. The scoped author groups used in downstream analyses are summarized in Table II. For footprint analyses, we also collect public metadata for 58,262 users who engage with agent PRs through comments, reviews, commits, and merge or close actions. Of these accounts, 56,190 have public contribution volume, with a median account tenure of 5.35 years (IQR 1.99–10.14).

B. Analysis subsets

After deduplication, the scoped corpus contains 325,523 agent-authored PRs, 29,808 human-authored PRs, and 1,206 bot-authored PRs. These groups form the common starting point for the analysis. All downstream constructs are derived from reconstructed public PR timelines, but the unit of analysis differs by research question. RQ1 and RQ4 use PR-level denominators for entry and integration outcomes. RQ2 uses feedback-event and feedback-body denominators. RQ3 uses both PR-level pathway denominators and cycle-level denominators for latency and response measures. We state the exact denominators where each result appears. This design avoids comparing event-level percentages with PR-level percentages and keeps PRs without feedback in pathway and endpoint denominators while excluding them only from measures that require feedback, response, or endpoint timestamps.

C. PR Timeline Construction

PR Timeline and Action Types. For each PR, we reconstruct an ordered public timeline by merging `Open`, `Comment`, `Review`, and `Commit` actions with normalized actor logins and UTC timestamps. The first `Open` action defines the opening time and normalized PR-author login. The resulting timeline is the ordered action sequence used through the paper.

The first entry event is defined as the earliest `Comment` or `Review` action from a non-author whose timestamp is strictly later than the opening time. We use *entry* for this first PR-thread contact so that feedback comments and reviews are not conflated with later code responses. Commits are handled separately as response-eligible code activity in RQ3. Actions authored by the PR author are excluded so that self-follow-up does not count as an entry. When multiple candidate actions share the same timestamp, ties are broken in the order `Comment` to `Review`. This rule yields a deterministic first-entry label when multiple conversational actions occur at the same recorded time.

The same rule is applied consistently to agent-, human-, and bot-authored PRs in the baseline comparison. PRs without

¹anonymous.4open.science/r/icse27_submission-0F26

TABLE I
REPOSITORY-VISIBLE SIGNALS AND UNIQUE COLLECTED DATASET SIZE BEFORE SCOPED BASELINE FILTERING.

Author	Repository-visible signal	Unique PRs	Base repos	Commits	Comments	Reviews
Google Jules	author:google-labs-jules[bot]	64,547	17,203	67,468	29,025	7,582
Cursor	head:cursor/	54,039	20,373	76,233	28,882	5,644
Devin	author:devin-ai-integration[bot]	49,305	7,092	99,741	56,576	13,029
OpenAI Codex	head:codex/	48,471	18,070	47,021	6,812	3,997
Claude Code	"Co-Authored-By: Claude"; "Generated with Claude Code"	46,772	14,832	131,415	35,941	20,674
GitHub Copilot	head:copilot/	44,886	20,153	108,167	42,226	31,672
JetBrains Junie	author:jetbrains-junie,junie-eap[bot]	10,472	9,596	17,018	1,345	270
Codegen	"About Codegen"	8,651	2,356	17,343	17,088	3,025
Cosine	head:cosine/	4,373	904	12,348	1,521	685
Amazon Q Developer	author:amazon-q-developer[bot]	2,129	684	2,534	7,308	10,385
OpenHands	head:openhands; {"Co-authored-by": "Automatic fix generated by"} "OpenHands"	1,797	340	6,529	3,423	1,556
Tembo	head:tembo/	1,024	197	1,629	903	144
Human	Same-window human baseline	31,443	22,538	95,424	18,559	21,401
Total		367,868	129,993	682,324	249,539	120,042

TABLE II
SCOPED PR ANALYSIS GROUPS BY PR AUTHOR CLASS. HERE BOT DENOTES THE BOT-AUTHORED BASELINE. RAW PR-THREAD ENTRY COUNTS PRs WHOSE FIRST POST-OPEN NON-AUTHOR PR-THREAD EVENT IS A COMMENT OR REVIEW.

Author	PRs	Merged	Raw PR-thread entry
Agent	325,523	224,269	86,504
Human	29,808	21,553	9,989
Bot	1,206	882	509

candidate entry events are excluded from first-entry actor measures but kept in the full PR corpus for pathway analyses.

The lower panel of Figure 1 illustrates the PR-timeline representation used throughout the paper and shows the re-construction procedure on a representative PR.

D. Baseline construction

Baseline sets are constructed from the same study window as the agent PR corpus (2025-04-01 to 2026-01-31) and processed with the same canonicalization pipeline. For all author groups, each PR is mapped to a canonical key of the form `repo_full_name pr_number`. When GraphQL pull-request identifiers are available, they are first aligned to this canonical key so that PR rows and PR-action rows can be joined consistently across source tables. After time-window filtering, duplicate PR rows are removed by canonical PR key, keeping the earliest open action for each PR.

The human baseline is drawn from the companion human PR corpus and is used only for cross-group comparison. The bot baseline is defined coarsely from normalized author logins that match common bot patterns, including explicit `[bot]` markers and dependency-update accounts such as Dependabot and Renovate. In this design, bot-authored PRs are grouped into a single bot baseline for cross-group comparison.

The three author groups hence share the same study window, PR-key construction, date filtering, and deduplication pipeline.

The cross-group analyses support the same-window comparison.

E. Human-Agent PR Workflow Model

Prior work studies modern code review, PR mining, automation in PR threads, and agent-authored PRs through signals such as reviews, code changes, latency, and merge outcomes. We connect these signals in one PR trajectory and use it to organize the four RQs and the measures below.

We model each PR as a Human-Agent PR workflow with four stages. They are *Entry*, *Feedback*, *Feedback-response Cycle*, and *Integration*. The stages are grounded in GitHub PR events. Entry begins with the PR `Open` event and tracks whether a later non-author PR-thread entry appears and who first triages the PR. Feedback uses PR `Comment` and `Review` events from humans, agents, or automation bots, and is characterized by actor, body form, and feedback intent. Feedback-response Cycle links feedback blocks to later response-eligible `Commit` events, measuring response timing, response actors, and code-change magnitude. Integration uses merge and close metadata to classify the PR as merged, closed without merge, or still open at data collection.

Figure 1 maps the four RQs onto this model, distinguishing PRs that share an endpoint but differ in triage, feedback, response, and revision.

F. Taxonomy Construction

1) *RQ1: Entry: Interaction Sequence.* To characterize how collaboration begins, we extract entry events from PR timelines. For each PR, raw entry is the earliest non-author PR-thread comment or review after opening. Retained entry applies the feedback filter described below before selecting the first retained event. Commits are not treated as entry feedback and are reserved for code-response analyses.

Entry Measures. The analysis reports raw and retained entry rates and first retained entry actor.

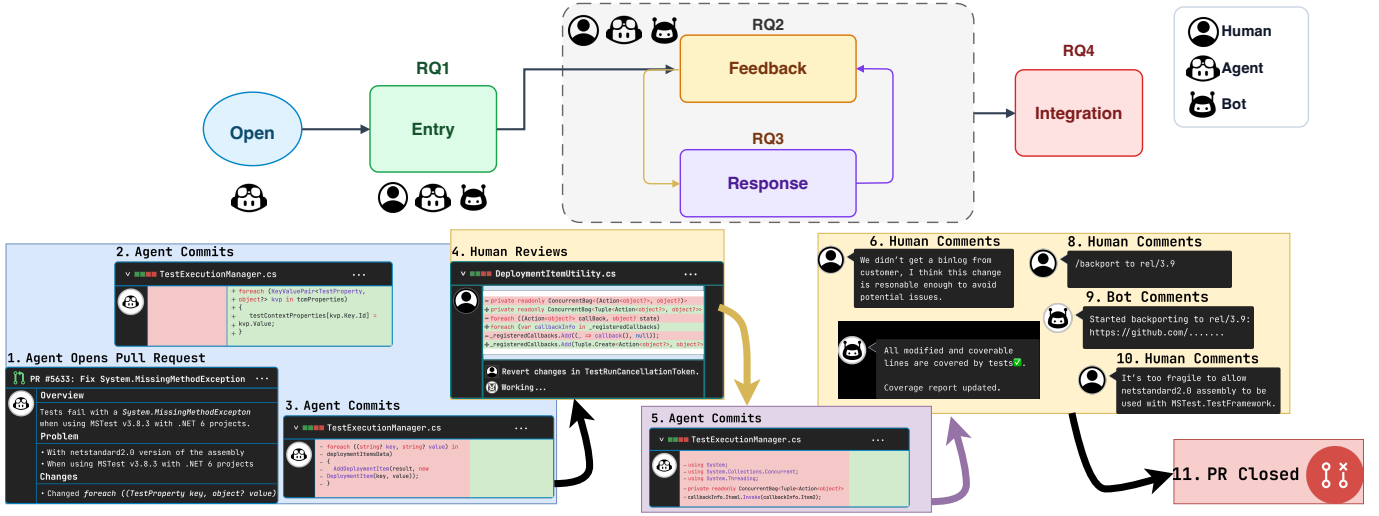


Fig. 1. Human-agent PR workflow model (upper) and example PR timeline for GitHub PR microsoft/testfx/pull/5633 (lower). Bot labels are automation bots.

Engaging Developers and Coverage. An engaging developer is defined as any account that performs at least one human PR action in an agent-authored PR during the study window, for example, commenting, reviewing, committing, or creating an issue. Accounts identified as non-human automation (bots) or coding agents are excluded using the tracking signals in Table I. The packaged user table contains 58,262 accounts with account-creation timestamps and contribution summaries of the past year preceding data collection, of which 56,190 have nonzero public contribution volume. For first-human-triager footprint analyses, we join PR-thread accounts to this public-profile table and report unmatched accounts as no-metadata.

Public GitHub Footprint Score. Following large-scale GitHub profiling work [18], we construct an empirical public GitHub footprint score for matched first human responders from two public profile signals, account tenure and cumulative public contribution volume. For each user u , $\text{tenure}(u)$ is the number of days between `created_at` and a fixed reference time, and $\text{contrib}(u) = \# \text{commits}(u) + \# \text{PRs}(u) + \# \text{reviews}(u) + \# \text{issues}(u)$. We apply $\log(1 + x)$ to both components, winsorize each transformed component at the 5th and 95th percentiles, and min-max normalize to $[0, 1]$. The main specification is an equal-weight score $\text{footprint}(u) = 0.5 \text{tenure}(u) + 0.5 \text{contrib}(u)$.

Footprint Strata. We convert the continuous score into empirical quartiles **Q1-4 footprint** within the covered user subset. For footprint-stratified PR-level analyses, each PR receives the stratum of its first matched non-author human responder. PRs without a matched responder remain unlabeled and are excluded from these summaries.

2) **RQ2: Feedback.** RQ2 characterizes the feedback that appears after an agent-authored PR enters the PR thread. Feedback events consist of non-author PR-thread comments and reviews after PR opening. Each event retains its event type, actor, timestamp, and available text body. We report comments and reviews separately where useful because short comments,

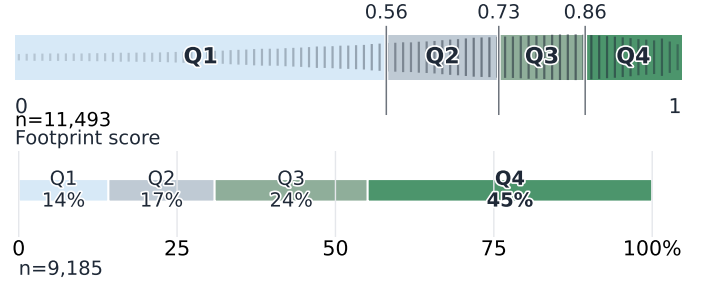


Fig. 2. Human footprint score quartiles and scoreable first-human triagers.

formal review bodies, and generated review reports can carry different kinds of feedback.

Feedback actors. Each feedback event receives the refined actor class defined above. Accounts marked by the coarse detector but not safely separable by the refined rules remain *unresolved agent/bot*. Missing or unrecognized actors remain *unknown*. Unlike RQ1, which focuses on the first retained triager per PR, RQ2 summarizes all eligible feedback events. When the feedback actor is a matched human, we attach the public-footprint stratum defined in RQ1.

Syntax features. Following work on code-review usefulness, review-comment classification, and review automation [19]–[24], we extract deterministic body-form features from each available feedback body before assigning intent labels. These features capture body length, code grounding through fenced or inline code markers, structured packaging through Markdown bullets, numbered lists, tables, or summary-like sections, and boilerplate/status markers for CI, deployment, routing, helper, or administrative messages.

Intent taxonomy. We construct an intent taxonomy for PR-thread feedback following the human-in-the-loop LLM taxonomy workflow [25], which uses an initial taxonomy, LLM-assisted labeling, human validation, and full-corpus application. Our initial taxonomy is seeded from prior work on

modern code review and review-comment classification. Code review is known to support defect finding, maintainability discussion, knowledge transfer, and integration decisions [26]–[28]. PR reviews and comments have also been classified by correctness, management, interaction, and other review functions [20]. Recent work on useful or LLM-generated code-review comments motivates distinguishing actionable concerns from non-actionable or generated feedback [19], [21]. Based on these sources and pilot inspection of our feedback bodies, we define seven primary intent labels for aggregate feedback summaries. They are *Functional*, *Maintainability*, *Documentation*, *Discussion*, *Process*, *Validation*, and *Non-actionable*. *Maintainability* covers refactoring and cleanup requests, *Process* covers integration, CI, merge, and workflow coordination, and *Validation* covers evidence, reproduction, test, or verification requests. Figures abbreviate labels only for space, such as *Maint.*, *Docs*, and *Non-action*.

Intent labeling. We use the codebook for LLM-assisted labeling. Two human annotators independently labeled 141 randomly sampled feedback events using the same primary-label taxonomy. After applying the edited disagreement table, 104 events had matching primary intent labels, yielding Cohen’s $\kappa = 0.683$, which indicates substantial agreement [29]. The remaining 37 disagreements are excluded from the consensus-gold evaluation. For the full-corpus labeling, we use the selected scalable setup of `openai/gpt-4o-mini` with batch size 20 and two examples per label. For this primary-intent labeling task, the 29-row held-out consensus set yields 0.690 accuracy, 0.577 macro-F1, 0.676 weighted-F1, and no invalid outputs. These labels are used only as an aggregate descriptive layer: they summarize what retained feedback appears to ask for, but they do not define entry, cycle, pathway, revision, or integration measures. The primary workflow claims rely on event type, actor class, timestamps, commits, and endpoint metadata. Given the small held-out set, intent-stratified contrasts should be read as descriptive patterns rather than standalone evidence about feedback quality or causal effects. We apply this setup to the remaining eligible feedback events.

3) *RQ3: Feedback-Response Cycles*: RQ3 asks whether commits follow feedback, how many cycles occur, how long they take, and how much code changes.

Blocks and cycles. A *feedback block* is a contiguous run of feedback events. A new block begins after an intervening commit. A *response block* is the set of commits after a feedback block and before the next feedback block. A *feedback-response cycle* exists when a feedback block has at least one following commit in its response block. If no later commit shows, the block is feedback without a code response.

Response-eligible commits occur after the PR is opened and before the PR is merged or closed. We exclude routine merge artifacts, merge-button or merge-queue boundary commits, and commits at or after terminal merge/close. Events are sorted by timestamp, event priority, and event ID. At identical timestamps, feedback is ordered before commits, so same-time feedback followed by a commit can be counted as a zero-

latency response.

Pathway taxonomy. Each scoped PR receives one feedback-response pathway. The pathway taxonomy is derived exclusively from the reconstructed PR timeline dataset. The rules are mutually exclusive and exhaustive for scoped PR timelines. Each PR has no visible feedback, feedback without a later code response, one response cycle, multiple response cycles, or terminal feedback after the final code change.

- *No visible feedback*. No feedback block appears.
- *Feedback without code response*. Feedback exists, but no later response-eligible commit follows.
- *Single-feedback response cycle*. Exactly one feedback block is followed by a response block.
- *Multi-feedback response cycle*. Two or more feedback blocks are followed by response blocks.
- *Late feedback after final code change*. The final feedback block starts after the final response block.

The late-feedback category takes precedence over the single- or multi-cycle count because it describes the PR’s final state. A PR in this category may yet have earlier cycles.

Cycle timing and code response. Cycle latency is the time from the end of a feedback block to the first commit in its response block. Latency summaries use only cycles with a response commit. Cycle times are skewed, so we report medians, IQRs, and upper percentiles.

We measure code-response size at two levels. At the cycle level, we count response commits, changed lines, changed files, and changed lines per feedback event for each response block. At the PR level, we split response-eligible commits into pre-feedback and post-feedback commits using the first feedback event. PRs without visible feedback, or without later response commits, remain in the PR-level denominator with zero post-feedback code response.

We measure changed lines as additions plus deletions when both are available. Changed files come directly from GitHub’s per-commit `changed_files`. When a stat is missing, we leave it missing and report how often it occurs.

Response actors. For response blocks, we retain the committer GitHub accounts and other activities following feedback.

4) *RQ4: Integration*: RQ4 asks how the PR workflows characterized in the earlier stages are associated with repository endpoints. Each PR receives one endpoint label: *merged* when a merge timestamp is present, *closed without merge* when a close timestamp appears without a merge timestamp, and *open* when no close timestamp appears by the end of data collection. When merge and close metadata are both present, we classify the PR as merged because merged PRs are also closed by GitHub workflow.

RQ4 keeps the full PR-level denominator, so PRs without visible feedback still receive endpoint labels. The primary stratification is the RQ3 feedback-response pathway, which summarizes whether feedback appeared, whether code activity followed, and whether final feedback was close- or merge-adjacent. Feedback-to-endpoint timing is computed only when first feedback strictly precedes merge or close; other PRs re-

main in endpoint denominators but are excluded from latency summaries. Open or without-close PRs are right-censored by the data-collection window and are interpreted as extraction-time states rather than final states. Endpoint contrasts use percentage point (pp) differences from the specified baseline. For construct-specific panels, the baseline is the endpoint rate over the construct’s covered PR subset before splitting by that construct. The covered subsets are first-retained-actor PRs for actor rows, first-retained-intent or body-form PRs for intent/body rows, and scoreable first-human-triager PRs for footprint rows. Reported deltas are stratified endpoint rates minus the construct baseline. No regression adjustment is implied.

G. Validation and Robustness

Retained-feedback sensitivity. We evaluate four feedback boundaries. *Raw visible feedback* includes all post-open non-author PR comments and reviews. *Relaxed retained feedback* removes only empty-body events and comments or reviews from known status, deployment, CI, helper, routing, release, security-scan, and administrative service accounts. *Main retained feedback* further excludes deployment previews, CI/check/build summaries, helper invitations, routing or assignment commands, generated summary shells without review findings, release/version notices, CLA/administrative messages, security-scan notices, and other boilerplate/status-only events matched by the retained-feedback codebook. We use this main retained boundary for retained-entry, feedback-actor, body-form, and intent summaries. Structural pathway analyses keep the broader visible-feedback timeline and state the denominator where results appear. The filter is content/status based, and agent or bot messages are retained if they contain substantive review, process, evidence, or change-request content. To sanity-check, the authors jointly audited a stratified sample of 100 retained and 100 excluded feedback events and found no systematic rule violations. *Strict retained feedback* additionally removes short non-actionable acknowledgments and terminal signals, such as approval-only, thanks-only, LGTM-only, no-comments/no-issues, and merge-ready comments, unless they contain actionable cues such as fix, test, validate, refactor, or failure-related requests. Across non-raw retained definitions, retained-feedback entry remains a minority of agent-authored PRs. Relaxed, main, and strict retained boundaries cover 13.3%, 10.5%, and 10.3% of agent-authored PRs, respectively.

Footprint-score sensitivity. We recompute footprint strata using tenure-only, contribution-only, and alternative tenure/contribution weights, while keeping each PR’s first-human-responder assignment fixed. PRs whose first human responder lacks scoreable metadata remain outside these footprint-stratified summaries.

Endpoint censoring sensitivity. Because open or without-close endpoints may reflect the collection boundary, we repeat integration summaries after excluding PRs opened near the end of the study window and after exclusions that leave one top agent out, so that endpoint patterns are not affected by

late-window censoring or a single dominant agent. Across the main and sensitivity summaries, proportions are reported with counts and denominators, with Wilson or difference intervals for selected actor, syntax, and endpoint contrasts in the artifact.

III. RESULTS

A. RQ1: Entry

Entry denominators. Among 325,523 agent-authored PRs, 86,504 (26.6%) have raw entry through a non-author comment or review, 34,049 (10.5%) retain review-relevant feedback after filtering, and 33,502 (10.3%) contain at least one feedback-response cycle. The retained-entry denominator is therefore much narrower than the raw visible PR-thread record, but it is the relevant denominator for asking who first enters the PR thread with substantive retained feedback.

First retained entry actor. In the 34,049 retained-entry PRs, the first retained actor is nearly split between humans (17,037 PRs, 50.0%, Wilson 95% CI [49.5, 50.6]) and agents (16,132 PRs, 47.4%, [46.8, 47.9]). Automation bots account for 856 PRs (2.5%, [2.4, 2.7]) and unresolved agent/bot accounts for 24 PRs (0.1%, [0.0, 0.1]). RQ1’s answer is therefore that retained PR-thread feedback occurs for a minority of agent-authored PRs. When it occurs, the first retained entry actor is almost evenly split between humans and agents.

First human triager footprint. Figure 2 stratifies the human side of RQ1 using an equal-weight account footprint score with 0.5 normalized account tenure and 0.5 normalized public contribution volume. The score-distribution plot shows the covered account distribution over this continuous score, with quartile cut points at 0.56, 0.73, and 0.86. Among the 17,037 PRs whose first retained entry came from a human, 9,185 (53.9%) join to a scoreable engaging account in the footprint table. The first-human-triager plot displays this scoreable subset. Q1 accounts provide the first retained human entry for 1,301 PRs (14.2%), Q2 accounts for 1,540 PRs (16.8%), Q3 accounts for 2,219 PRs (24.2%), and Q4 accounts for 4,125 PRs (44.9%). The remaining 7,852 first-human PRs (46.1% of the full first-human denominator) come from accounts whose metadata is unavailable and are therefore not assigned to quartiles. These no-metadata PRs correspond to 2,809 unique first-human triager accounts that appear in the retained-feedbacks but have no match in the user metadata and footprint-score table. The no-metadata stratum denotes uncovered metadata. Thus, among the 53.9% of first-human retained-entry PRs with scoreable metadata, developers in the largest footprint quartile account for 44.9% of first human entries.

B. RQ2: Feedback

Feedback channels and syntax features. Figure 3 summarizes the channel and body-shape side of RQ2. The RQ2 denominator is 76,936 retained feedback events with deduplicated event IDs. Most retained feedback appears as PR comments, with 56,589 events (73.6%). GitHub review events account for the remaining 20,347 events (26.4%). Figure 3 shows that feedback on agent-authored PRs is mostly thread comment feedback rather than review events.

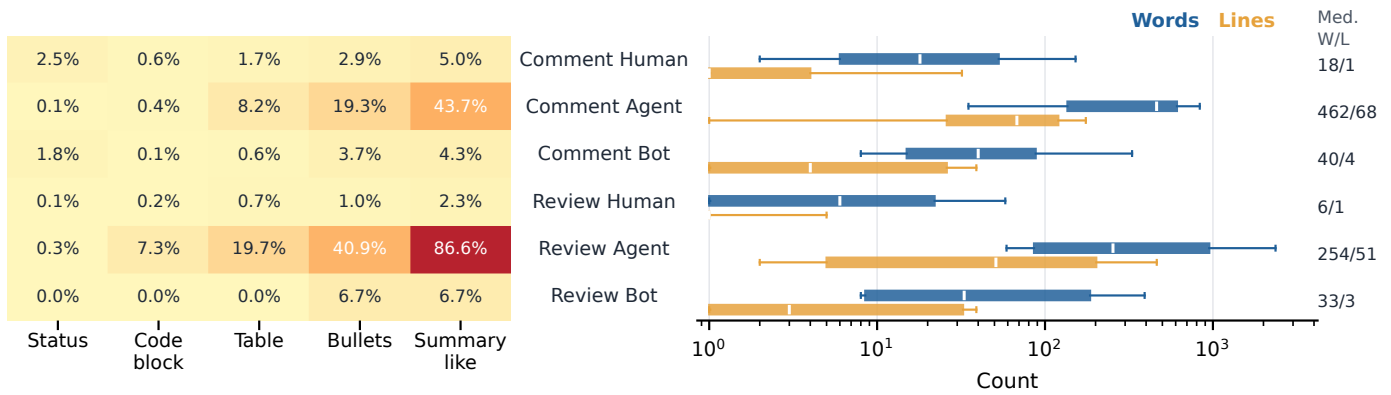


Fig. 3. RQ2 feedback body form by channel and actor. Left heatmap cells report the percentage of retained feedback events in each channel-actor group whose body matches each syntax feature. Right boxplots summarize feedback-body word and line counts on a log scale. Boxes show the IQR, whiskers P10–P90, numeric labels mark medians. Outliers hidden. Bot=automation bot.

The body text also differs strongly by actor. Human feedback is usually short. Human comments have a median of 18 words and one line, while human reviews have a median of 6 words and one line. The one-line pattern for human reviews is real in the data because the 75th percentile is still one line, and the 90th percentile is five lines. Agent feedback is much longer. Agent comments have a median of 462 words and 68 lines, while agent reviews have a median of 254 words and 51 lines.

The syntax features tell the same story in another way. Human feedback rarely uses structured markup features such as code blocks, tables, bullets, or summary-like format. Agent feedback uses these features much more often, notably in review events. Compared with human reviews, agent reviews are +84.3 pp higher on summary-like form (95% CI [+83.6, +85.0]) and +39.9 pp higher on bullets ([+39.0, +40.9]); in comments, the summary-like gap is +38.6 pp ([+38.0, +39.3]). Figure 3 also shows that comments are the larger feedback channel, and agent feedback is more structured.

Feedback actors. The top row of Figure 4 answers the actor part of RQ2 across all retained feedback and by channel. On all retained feedback, agents provide 35,115 events (45.6%). Human accounts provide 40,231 events (52.3%), and automation bots provide 1,590 (2.1%). This all-event view differs from RQ1 because it asks who enters first, while RQ2 asks who provides the feedback events that appear after entry.

The actor mix changes by channel. Agents provide 11,188 of 20,347 reviews (55.0%) versus 23,927 of 56,589 comments (42.3%), a +12.7 pp difference between reviews and comments (95% CI [+11.9, +13.5]). Automation bots provide 2.8% of comments but only 0.1% of reviews. Figure 4 shows review events are more agent-heavy than comments.

The human side is not evenly distributed across footprint strata. Among humans, Q1 provides 3,682 retained feedback events (4.8% of all retained feedback events), Q2 provides 3,362 events (4.4%), Q3 provides 5,525 events (7.2%), and Q4 provides 10,977 events (14.3%). Another 16,685 events (21.7%) come from humans without footprint metadata. As in RQ1, this no-metadata group denotes accounts that are not

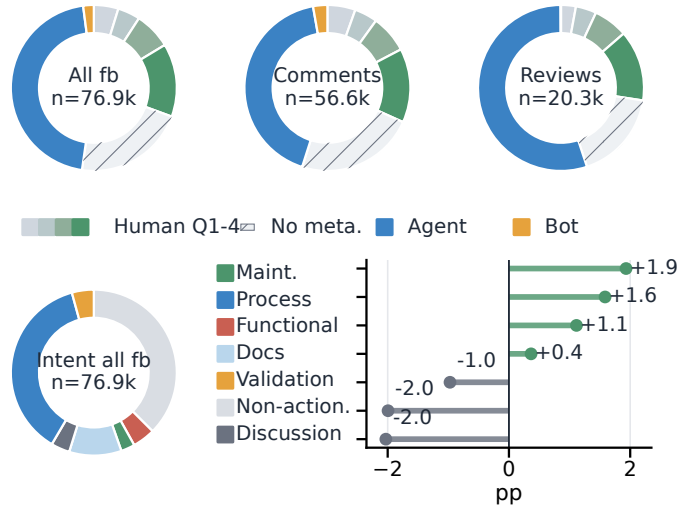


Fig. 4. RQ2 event-level feedback actors and intent mix for 76,936 retained feedback events. Panels show actor shares, primary-intent shares, and review-minus-comment intent differences in pp. Maint.=Maintainability, Docs=Documentation, Non-action.=Non-actionable, Bot=automation bot.

able to join the footprint-score table. Within the human strata, Q4 accounts have the majority of human feedback.

Feedback intent. The bottom row of Figure 4 summarizes intent labels attached to feedback events. The overall distribution is concentrated in a few intent types. Non-actionable feedback accounts for 28,837 events (37.5%). Process feedback has 28,683 events (37.3%). Documentation feedback is the next largest group with 7,836 events (10.2%). The remaining intent groups are smaller than 5% of all feedback events.

The comparison between reviews and comments is more stable than the actor and body-shape comparison. Every intent category appears in both comments and reviews, and their shares are similar. For reviews, non-actionable and process feedback account for 36.0% and 38.4%. For comments, they account for 38.0% and 36.9%. The largest absolute channel differences are about two percentage points (pp). Discussion feedback is lower in reviews than in comments, Maintainabil-

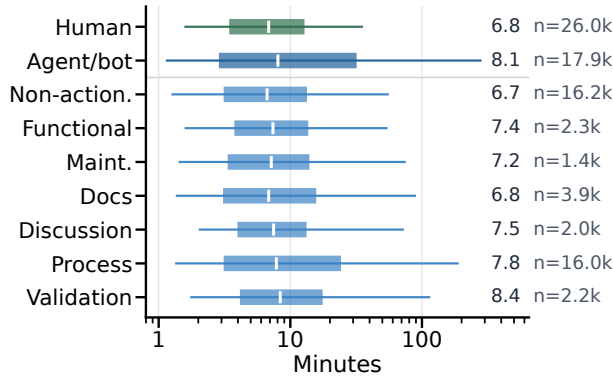


Fig. 5. Cycle-level feedback-response latency for 59,159 visible feedback-response cycles across 33,502 agent-authored PRs. Actor rows stratify cycles by feedback actor. Agent/bot combines agent accounts and automation bots. Intent rows pool cycles across actors and stratify by primary feedback intent. Thick bands show IQR, whiskers P10–P90, white ticks/numbers medians, and right labels row counts.

ity feedback is higher in reviews, and Process feedback is slightly higher in reviews. These differences are much smaller than the actor and syntax differences shown above.

Figures 3 and 4 show that retained feedback differs most by channel, actor, and body form. Intent differences between comments and reviews are smaller.

C. RQ3: Feedback-response Cycles

Workflow pathway composition. Over all, most agent-authored PRs do not enter a feedback-response pathway. There are 239,019 PRs (73.4%) with no visible feedback. Another 44,177 PRs (13.6%) receive feedback but have no later response-eligible commit. Explicit response cycles are less common. Only 16,446 PRs (5.1%) have one feedback-response cycle, and 4,682 PRs (1.4%) have multiple cycles. The remaining 21,199 PRs (6.5%) end with late feedback after the final code change. Thus, RQ3 shows that response cycles are uncommon. Most PRs have no visible feedback or feedback without later code response.

Cycle latency. Among agent-authored PRs with feedback-response cycles, the latency table contains 59,159 cycles across 33,502 PRs. The median cycle latency is 7.0 minutes. Only 54 of 59,159 cycles (0.09%) have zero latency from same-timestamp ordering, so the 7.0-minute median is not driven by the tie-breaking rule. The majority of cycles are fast, with 62.1% occurring within 10 minutes and 88.9% occurring within 60 minutes. The distribution is however long-tailed, with P90 at 76.1 minutes and P95 at 483.2 minutes. The retained-feedback cycle subset is similar at the median. It contains 31,394 retained cycles across 16,775 PRs with a median latency of 7.1 minutes and P90 of 65.0 minutes. Because cycle latency is skewed, we emphasize medians and upper percentiles rather than averages. Figure 5 visualizes the skew with compact log-scale interval strips. Figure 5 shows the same median-fast, tail-heavy pattern by actor and intent.

Response. In the 59,159 agent-authored feedback-response cycles, response commit accounts are often not limited to

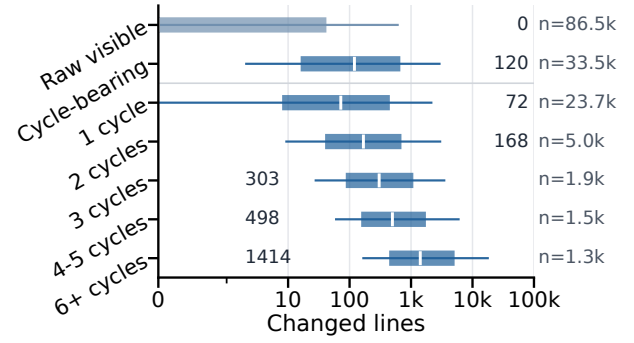


Fig. 6. Code-response magnitude after visible feedback. Bands show IQR, whiskers P10–P90, white ticks/numbers mark medians. Outliers are hidden.

the PR author. The largest response-actor category is PR author + others, covering 25,825 cycles (43.7%). Mixed non-author response accounts cover another 19,848 cycles (33.6%). Non-author human-only response blocks account for 7,955 cycles (13.4%), while PR-author-only response blocks account for 4,290 cycles (7.3%). Non-author agent/bot-only response blocks are rather rare at 1,241 cycles (2.1%). The five strata are mutually exclusive. They are PR-author-only, PR author plus other accounts, non-author human-only, non-author agent/bot-only, and mixed non-author accounts with both human and agent/bot non-author commit accounts. These categories report commit accounts attached to GitHub commits.

Figure 6 reports the code-response magnitude after visible feedback. For PRs with raw visible feedback, changed lines remain sparse (median 0; P90 631; P95 2,074), but the feedback-with-code-response subset is larger at the median and tail (median 120; P90 3,024; P95 8,164). The cycle-count strata show that repeated cycles are associated with larger response. One-cycle PRs have a median of 72 post-feedback changed lines, whereas PRs with six or more cycles have one of 1,414 changed lines. The upper tail also grows with cycle count, from P95 of 6,010 changed lines for one-cycle PRs to P95 of 49,791 changed lines for PRs with six or more cycles. The outlier check reports changed-lines P95 of 8,164 in the visible-feedback and code-response denominator, 6,440 after dropping the top 1%, and 3,407 after dropping PRs above 10,000 changed lines. There is hence a two-level pattern. Many raw-visible-feedback PRs have no measured post-feedback code response, while cycle-bearing PRs have larger, heavy-tailed responses.

D. RQ4: Integration

Merge outcomes over the entry-cycle composition. Figure 7 composes the RQ1 entry and RQ3 feedback-response pathway summaries with integration endpoints. The unit of analysis is the agent-authored PR, and each is assigned one endpoint category. The endpoint categories are merged, closed without merging, and open PR. The pathway-specific endpoint percentages use the same PR-level denominators introduced in RQ3. These denominators are 239,019 no-visible-feedback PRs, 44,177 feedback-without-code-response PRs, 16,446 single-

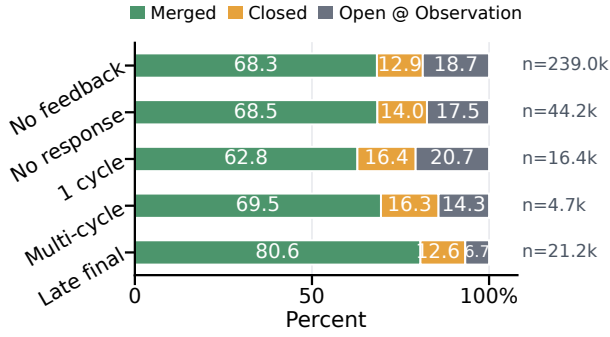


Fig. 7. PR-level RQ4 integration endpoint composition by feedback-response pathway for 325,523 agent-authored PRs. Each PR is assigned one endpoint (merged, closed without merge, or open PR at observation). Each row sums to 100%, and right labels report pathway denominators.

cycle PRs, 4,682 multi-cycle PRs, and 21,199 late-feedback PRs. No-visible-feedback PRs are the pathway reference, with a 68.3% merge rate. Relative to that reference, feedback-without-code-response PRs are nearly unchanged (+0.1 pp, 95% CI [-0.3, +0.6]), single-cycle PRs are lower (-5.5 pp, [-6.3, -4.7]), multi-cycle PRs are similar (+1.1 pp, [-0.2, +2.5]), and late-feedback PRs are higher (+12.3 pp, [+11.7, +12.9]); single-cycle PRs also have the largest open PR share (20.7%).

Terminal feedback as an integration signal. The late-feedback pathway needs a terminal interpretation because its final feedback block starts after the final code change and has no later response block. The terminal diagnostic breakdown supports this interpretation: 11,940 late-feedback PRs end with a review-only final feedback block and merge at 89.0%; another 8,282 end with an APPROVED-only review state and merge at 93.8%. Among merged late-feedback PRs, 3,536 record last feedback after the merge timestamp and 9,371 within five minutes before merge. This pattern indicates that late feedback often records approval-like or merge-adjacent integration evidence. The body-form endpoint breakdown is consistent with that interpretation: summary-like first retained bodies are 10.7 pp above the body-form baseline on merged, while short and long plain-text bodies are 4.2 and 3.6 pp below it. For open PR endpoints, the pattern reverses: summary-like first retained bodies are 4.8 pp below the body-form baseline, while short and long plain-text bodies are 2.0 and 1.6 pp above it.

Integrated interpretation. Figure 8 reports merged-rate differences from construct-specific baselines for selected actor, intent, and first-human footprint-quartile strata. For first retained actor, the agent row is 2.8 pp above the actor baseline on merged, while the human row is 2.8 pp below it. For the first retained intent, Process and Maintainability rows are 3.9 and 4.5 pp above the intent baseline on merged, while Documentation and Discussion are 8.6 and 8.2 pp below it. For scoreable first-human triagers, Q1 and Q3 are 2.3 and 0.9 pp below the footprint baseline on merged, while Q2 and Q4 are 0.1 and 1.2 pp above it. For open PR endpoints, the breakdown behind Figure 8 shows the largest positive contrasts in documentation-first PRs and Q1 first-human triagers: +13.4

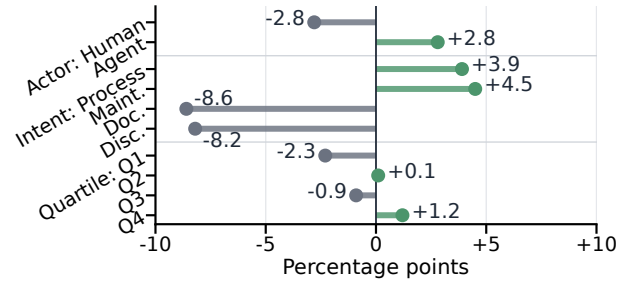


Fig. 8. PR-level RQ4 selected merged-rate differences from construct-specific baselines. Zero is the merged rate within each construct’s covered subset before stratification (first retained actor, first retained intent/body form, or scoreable first-human triagers). Points are pp differences from that baseline.

pp from the intent baseline and +7.0 pp from the footprint baseline, respectively. Summary-like first retained bodies are 4.8 pp below the body-form baseline on open PR, and Q4 first-human triagers are 3.9 pp below the footprint baseline. The Q1 row has the smallest quartile denominator (1,301 PRs). Together, Figures 7 and 8 show that integration endpoints vary with the preceding workflow pathway, so similar merge labels can hide different entry, feedback, and response-cycle states.

IV. DISCUSSION

We study agent-authored PRs as engagement workflows, not as acceptance outcomes. The results do not collapse to one signal. Retained feedback, response cycles, revision, and integration capture different parts of the PR path. A raw “reviewed” or “merged” label can therefore mix PRs with no retained feedback, feedback without response, iterative cycles, terminal approval-like feedback, or larger revisions.

A. Implications for the Agent Development Lifecycle (ADLC)

Existing ADLC discussions give names to evaluation, observability, and guardrails around agents [30], [31]. Our data add the repository boundary. An agent patch has not finished its lifecycle when a PR opens. It still has to engage with, or fail to engage with public feedback, actor participation, code response, and integration. Only 26.6% of agent-authored PRs receive raw visible feedback, retained review-relevant feedback remains 10.3% to 13.3% across filtering boundaries, and retained-feedback first entry is nearly split between humans and agents. For ADLC, observability should cover repository uptake as well as agent-internal execution: which generated changes receive review-relevant feedback, who provides it, and whether later PR activity produces code response.

B. Workflow Metrics for Agent-authored Software Changes

The pathway results explain why merge should not stand alone. A merged agent-authored PR may have no visible feedback, feedback without code response, one response cycle, repeated cycles, or terminal merge-adjacent feedback. These states point to different review and verification states, but they now collapse under one endpoint label. Integration should therefore be read together with the preceding pathway to see

whether feedback appeared, whether a code response followed, and whether the final feedback block was terminal.

C. Threats to Validity and Future Work

The corpus prioritizes precision and repository-visible signals. Following prior agent-authored PR mining work [6]–[8], the study identifies agent-authored PRs from public account, branch, and text signals. This improves attribution precision, but it cannot recover agent-assisted work hidden by private settings, disabled signals, or human accounts. The same issue may affect the human baseline, and non-repository-matched author groups make cross-group and endpoint contrasts descriptive rather than causal.

The workflow reconstruction captures public PR-thread traces: comments, reviews, commits, endpoints, and timestamps. It omits private review, local agent iteration, IDE/chat traces, and agent-runtime state. Feedback-response cycles and post-feedback revision are temporal signals; fuller traces would be needed to establish causality. Body-form and terminal-review endpoint patterns may also reflect repository automation and review conventions.

The retained-feedback filter and public GitHub footprint score also limit the constructs. Retained feedback is a conservative review-relevant subset, not a complete measure of all useful feedback. Footprint strata summarize public trace profiles rather than expertise or quality, and no-metadata accounts remain uncovered. Two endpoint robustness checks are stable: pathway endpoint rates are unchanged under the 30-day mature-PR filter, and late-feedback merge rates remain 79.1–81.1% in top-agent leave-one-out checks. Null-hypothesis significance testing is kept out of the main claims, while interval and effect-size outputs remain in the artifact. The findings are most directly informative for public GitHub repositories during 2025–2026 where agent activity is visibly marked and PR-based development is routine. Future work can connect these workflows to downstream maintenance signals and richer IDE, chat, CI, and agent-runtime traces.

V. RELATED WORK

a) *Coding Agents in Software Development:* AI support for programming has moved from local completion and chat assistance toward tools that can plan tasks, edit multiple files, run checks, and interact with repository infrastructure [2], [3], [32]. Benchmark studies still emphasize task completion, benchmark accuracy, or synthetic issue resolution [4], [5]. These studies measure coding capability more than repository uptake after an agent-produced change becomes a PR, while recent work shows that human feedback remains important during agent-assisted planning and code generation [33]. This study examines the later repository-facing stage after PR creation.

b) *Pull Requests as Socio-Technical Workflows:* Pull requests are socio-technical workflows, not only containers for code changes, and are shaped by technical risk, contributor reputation, project norms, and reviewer workload [15], [16]. Prior work links PR latency to contributor familiarity, project

load, and PR characteristics [17], and shows that review and acceptance outcomes depend on technical and social factors rather than code features alone [34]. Recent work also studies PRs as coordination structures, including overdue-PR interventions and links between PRs and issues [35], [36]. Mining studies warn that large-scale GitHub analyses inherit missing actions, noisy account labels, and selection bias [37]; this motivates the construct limits in Section IV-C.

c) *Review Automation and Human-Agent Feedback:* Modern PR threads contain human review, automation-bot messages, and increasingly agent-mediated review. Code-review bot research shows that automation can alter pull-request dynamics, developer workflows, and notification burden [22]–[24]. Large-scale evidence from review conversations shows that human reviewers provide broader feedback than agents, including understanding, testing, and knowledge transfer, and that adoption differs for AI and human suggestions [38]. This work motivates the actor layer, which separates humans, agents, and automation bots before analyzing response cycles, revision, and integration.

d) *Empirical Studies of Agent-Authored Pull Requests:* Recent empirical studies have made agent-authored PRs a rapidly growing target [1], [39]. Some characterize acceptance, merge outcomes, activity patterns, code change over time, and broad attributes [6], [8], [12]. Others focus on code modifications and PR-description consistency [9], test inclusion [10], security-related PRs [11], failed or rejected PRs [40], and cross-PR references [41].

The closest work examines review and collaboration signals around agent-authored PRs, including PR descriptions, human responses, reviewer engagement, actionable loops, and minimal-feedback merges [7], [13], [14]. This paper complements that cluster by reconstructing raw and retained feedback entry, first-entry actor, feedback actor, and intent, response-cycle latency, post-feedback revision, and pathway-conditioned integration endpoints.

VI. CONCLUSIONS

We reframe agent-authored PRs as engagement workflows rather than binary outcomes. Only 26.6% of 325,523 agent-authored PRs receive any visible feedback, and just 10.5% carry feedback substantive enough to retain under our main filter. Where retained feedback appears, triage is nearly split between humans and agents, meaning close to half of the first-line review of agent work is itself performed by agents, a finding that merge rate alone cannot surface. The pathway decomposition shows why this matters. No-feedback and feedback-without-response PRs together account for 87.0% of the corpus, yet the remaining pathways diverge sharply. A “merge” label conflates PRs that were rubber-stamped, silently integrated, or genuinely revised, states that matter for trust and oversight but are invisible at the endpoint. By making entry, retained feedback, response cycles, and integration separately measurable, our framework offers the ADLC community a way to ask not just whether agent contributions are merged but also how.

REFERENCES

- [1] H. Li, H. Zhang, and A. E. Hassan, "The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering," Jul. 2025, arXiv:2507.15003 [cs] version: 1. [Online]. Available: <http://arxiv.org/abs/2507.15003>
- [2] C. Treude and M. A. Gerosa, "How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering," Jan. 2025, arXiv:2501.08774 [cs] version: 1. [Online]. Available: <http://arxiv.org/abs/2501.08774>
- [3] V. Chen, A. Talwalkar, R. Brennan, and G. Neubig, "Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows," Sep. 2025, arXiv:2507.08149 [cs]. [Online]. Available: <http://arxiv.org/abs/2507.08149>
- [4] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, Dec. 2024, arXiv:2308.11432 [cs]. [Online]. Available: <http://arxiv.org/abs/2308.11432>
- [5] S. Liang, S. Garg, and R. Z. Moghaddam, "The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason," Dec. 2025, arXiv:2506.12286 [cs]. [Online]. Available: <http://arxiv.org/abs/2506.12286>
- [6] M. Watanabe, H. Li, Y. Kashiwa, B. Reid, H. Iida, and A. E. Hassan, "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub," Feb. 2026, arXiv:2509.14745 [cs]. [Online]. Available: <http://arxiv.org/abs/2509.14745>
- [7] K. Watanabe, R. Tsuchida, T. Monno, B. Huang, K. Yamasaki, Y. Fan, K. Shimari, and K. Matsumoto, "How AI Coding Agents Communicate: A Study of Pull Request Description Characteristics and Human Review Responses," Feb. 2026, arXiv:2602.17084 [cs]. [Online]. Available: <http://arxiv.org/abs/2602.17084>
- [8] R. M. Popescu, D. Gros, A. Botocan, R. Pandita, P. Devanbu, and M. Izadi, "Investigating autonomous agent contributions in the wild: Activity patterns and code change over time," in *Proceedings of the 23rd international conference on mining software repositories (MSR)*. New York, NY, USA: ACM, 2026, accepted to MSR 2026; arXiv:2604.00917. [Online]. Available: <https://arxiv.org/abs/2604.00917>
- [9] D. Ogenrwot and J. Businge, "How AI Coding Agents Modify Code: A Large-Scale Study of GitHub Pull Requests," Jan. 2026, arXiv:2601.17581 [cs]. [Online]. Available: <http://arxiv.org/abs/2601.17581>
- [10] S. Haque, S. Ingale, and C. Csallner, "Do Autonomous Agents Contribute Test Code? A Study of Tests in Agentic Pull Requests," Jan. 2026, arXiv:2601.03556 [cs]. [Online]. Available: <http://arxiv.org/abs/2601.03556>
- [11] M. L. Siddiq, X. Zhao, V. C. Lopes, B. Casey, and J. C. S. Santos, "Security in the Age of AI Teammates: An Empirical Study of Agentic Pull Requests on GitHub," Jan. 2026, arXiv:2601.00477 [cs]. [Online]. Available: <http://arxiv.org/abs/2601.00477>
- [12] H. Yoshioka, T. Monno, H. Tokumasu, T. Wakamatsu, Y. Ota, N. Weeraddana, and K. Matsumoto, "Let's Make Every Pull Request Meaningful: An Empirical Analysis of Developer and Agentic Pull Requests," Jan. 2026, arXiv:2601.18749 [cs]. [Online]. Available: <http://arxiv.org/abs/2601.18749>
- [13] C. Nachuma and M. Zibran, "When AI Teammates Meet Code Review: Collaboration Signals Shaping the Integration of Agent-Authored Pull Requests," Feb. 2026, arXiv:2602.19441 [cs]. [Online]. Available: <http://arxiv.org/abs/2602.19441>
- [14] H. Gao, P. Banyongrakul, H. Guan, M. Zahedi, and C. Treude, "On Autopilot? An Empirical Study of Human-AI Teaming and Review Practices in Open Source," Jan. 2026, arXiv:2601.13754 [cs]. [Online]. Available: <http://arxiv.org/abs/2601.13754>
- [15] J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of social and technical factors for evaluating contribution in GitHub," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE 2014. New York, NY, USA: Association for Computing Machinery, May 2014, pp. 356–366. [Online]. Available: <https://dl.acm.org/doi/10.1145/2568225.2568315>
- [16] O. Kononenko, T. Rose, O. Baysal, M. Godfrey, D. Theisen, and B. de Water, "Studying pull request merges: a case study of shopify's active merchant," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '18. New York, NY, USA: Association for Computing Machinery, May 2018, pp. 124–133. [Online]. Available: <https://dl.acm.org/doi/10.1145/3183519.3183542>
- [17] Y. Yu, H. Wang, V. Filkov, P. Devanbu, and B. Vasilescu, "Wait for It: Determinants of Pull Request Evaluation Latency on GitHub," in *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. Piscataway, NJ, USA: IEEE, May 2015, pp. 367–371, ISSN: 2160-1860. [Online]. Available: <https://doi.org/10.1109/MSR.2015.42>
- [18] L. Bao, X. Xia, D. Lo, and G. C. Murphy, "A Large Scale Study of Long-Time Contributor Prediction for GitHub Projects," *IEEE Transactions on Software Engineering*, vol. 47, no. 6, pp. 1277–1298, Jun. 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/8721092>
- [19] M. M. Rahman, C. K. Roy, and R. G. Kula, "Predicting usefulness of code review comments using textual features and developer experience," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. Piscataway, NJ, USA: IEEE, May 2017, pp. 215–226. [Online]. Available: <https://dl.acm.org/doi/10.1109/MSR.2017.17>
- [20] Z. Li, Y. Yu, G. Yin, T. Wang, Q. Fan, and H. Wang, "Automatic Classification of Review Comments in Pull-based Development Model," in *Proceedings of the 29th International Conference on Software Engineering and Knowledge Engineering (SEKE)*. United States: Knowledge Systems Institute Graduate School, Jul. 2017, pp. 572–577. [Online]. Available: <https://doi.org/10.18293/SEKE2017-039>
- [21] S. Goldman, H. Y. Lin, J. Pasuksmit, P. Thongtanunam, K. Tantithamthavorn, Z. Wang, R. Zhang, A. Behnaz, F. Jiang, M. Siers, R. Jiang, M. Buller, M. Jeong, and M. Wu, "What Types of Code Review Comments Do Developers Most Frequently Resolve?" Oct. 2025, arXiv:2510.05450 [cs]. [Online]. Available: <http://arxiv.org/abs/2510.05450>
- [22] M. Wessel, I. Wiese, I. Steinmacher, and M. A. Gerosa, "Don't Disturb Me: Challenges of Interacting with SoftwareBots on Open Source Software Projects," *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW2, pp. 1–21, Oct. 2021, arXiv:2103.13950 [cs]. [Online]. Available: <http://arxiv.org/abs/2103.13950>
- [23] M. Wessel, A. Serebrenik, I. Wiese, I. Steinmacher, and M. A. Gerosa, "Effects of Adopting Code Review Bots on Pull Requests to OSS Projects," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. United States: IEEE Computer Society, Sep. 2020, pp. 1–11, ISSN: 2576-3148. [Online]. Available: <https://ieeexplore.ieee.org/document/9240622>
- [24] —, "Quality Gatekeepers: Investigating the Effects of Code Review Bots on Pull Request Activities," *Empirical Software Engineering*, vol. 27, no. 5, p. 108, Sep. 2022, arXiv:2103.13547 [cs]. [Online]. Available: <http://arxiv.org/abs/2103.13547>
- [25] C. Shah, R. W. White, R. Andersen, G. Buscher, S. Counts, S. S. S. Das, A. Montazer, S. Manivannan, J. Neville, X. Ni, N. Rangan, T. Safavi, S. Suri, M. Wan, L. Wang, and L. Yang, "Using Large Language Models to Generate, Validate, and Apply User Intent Taxonomies," 2023, arXiv:2309.13063 [cs.IR]. [Online]. Available: <https://arxiv.org/abs/2309.13063>
- [26] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *2013 35th International Conference on Software Engineering (ICSE)*. Piscataway, NJ, USA: IEEE, May 2013, pp. 712–721. [Online]. Available: <https://ieeexplore.ieee.org/document/6606617>
- [27] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: a case study at google," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '18. New York, NY, USA: Association for Computing Machinery, May 2018, pp. 181–190. [Online]. Available: <https://dl.acm.org/doi/10.1145/3183519.3183525>
- [28] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "An empirical study of the impact of modern code review practices on software quality," *Empirical Software Engineering*, vol. 21, no. 5, pp. 2146–2189, Oct. 2016. [Online]. Available: <https://doi.org/10.1007/s10664-015-9381-9>
- [29] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. [Online]. Available: <https://journals.sagepub.com/doi/10.1177/001316446002000104>
- [30] D. Bergmann, "What is the agent development lifecycle (ADLC)?" May 2026, last modified May 11, 2026. Accessed: June 30, 2026. [Online]. Available: <https://www.ibm.com/think/topics/agent-development-lifecycle-adlc>
- [31] A. Osmani, S. Saboo, and S. Kartakis, "The New SDLC with Vibe Coding," Jun. 2026, published June

- 16, 2026. Accessed: June 30, 2026. [Online]. Available: <https://www.kaggle.com/whitepaper-the-new-SDLC-with-vibe-coding>
- [32] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot," Feb. 2023, arXiv:2302.06590 [cs]. [Online]. Available: <http://arxiv.org/abs/2302.06590>
- [33] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-In-the-Loop Software Development Agents," Jan. 2025, arXiv:2411.12924 [cs]. [Online]. Available: <http://arxiv.org/abs/2411.12924>
- [34] T. Dey and A. Mockus, "Effect of Technical and Social Factors on Pull Request Quality for the NPM Ecosystem," in *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, ser. ESEM '20. New York, NY, USA: Association for Computing Machinery, Oct. 2020, pp. 1–11. [Online]. Available: <https://dl.acm.org/doi/10.1145/3382494.3410685>
- [35] C. Maddila, S. S. Upadrasta, C. Bansal, N. Nagappan, G. Gousios, and A. v. Deursen, "Nudge: Accelerating Overdue Pull Requests Towards Completion," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 2, pp. 1–30, Apr. 2023, arXiv:2011.12468 [cs]. [Online]. Available: <http://arxiv.org/abs/2011.12468>
- [36] C. R. B. de Souza, E. Ma, J. Wong, D. Yoon, and I. Beschastnikh, "Revealing Software Development Work Patterns with PR-Issue Graph Topologies," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 106:2402–106:2423, Jul. 2024. [Online]. Available: <https://dl.acm.org/doi/10.1145/3660813>
- [37] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ser. MSR 2014. New York, NY, USA: Association for Computing Machinery, May 2014, pp. 92–101. [Online]. Available: <https://dl.acm.org/doi/10.1145/2597073.2597074>
- [38] S. Zhong, S. Noei, Y. Zou, and B. Adams, "Human-AI Synergy in Agentic Code Review," Mar. 2026, arXiv:2603.15911 [cs]. [Online]. Available: <http://arxiv.org/abs/2603.15911>
- [39] H. Li, H. Zhang, and A. E. Hassan, "AIDev: Studying AI Coding Agents on GitHub," Feb. 2026, arXiv:2602.09185 [cs.SE]. [Online]. Available: <http://arxiv.org/abs/2602.09185>
- [40] R. Ehsani, S. Pathak, S. Rawal, A. Mujahid, M. M. Imran, and P. Chatterjee, "Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub," Jan. 2026, arXiv:2601.15195 [cs.SE]. [Online]. Available: <http://arxiv.org/abs/2601.15195>
- [41] I. Khemissi, M. Chouchen, D. Wang, and R. G. Kula, "Humans Integrate, Agents Fix: How Agent-Authored Pull Requests Are Referenced in Practice," Apr. 2026, arXiv:2604.04059 [cs.SE]. [Online]. Available: <http://arxiv.org/abs/2604.04059>